

实验报告

课程:机器学习	实验名称:聚类	日期:6.1
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握聚类方法的原理，以及使用聚类方法求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法。
2. 对给定数据集，设计有关的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 聚类问题与可视化（模仿实例例题求解）

参考课本的实例例题，对无标签的鸢尾花（Iris）数据集，分别采用 k-均值和层次聚类算法发现数据中的簇，评估聚类结果的质量，并将聚类结果可视化。注：鸢尾花（Iris）数据集移除其中的类标签后，视为无标签的数据集。

1. 简单写出你的求解设计（例如：如何处理原始数据、如何进行聚类结果质量的评估、何种聚类方法更优、聚类结果如何可视化）。
2. 获得的结果，即聚类质量指标值、聚类结果的可视化。
3. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1. 求解设计

(1) 数据处理：加载Iris数据集（150个样本，4个特征：花萼长宽、花瓣长宽），移除类别标签将其视为无标签数据。使用StandardScaler对特征进行标准化（均值为0，标准差为1），消除量纲差异对距离计算的影响。

(2) 聚类方法：分别使用K-means (k=3, random_state=42, n_init=10) 和层次聚类 (AgglomerativeClustering, Ward最小方差法, n_clusters=3) 进行聚类。Ward方法通过最小化簇内方差来合并簇，效果通常优于单链接和完全链接。

(3) 质量评估：使用三个指标评估聚类质量——轮廓系数（取值 $[-1,1]$ ，越大越好，衡量样本与自身簇和最近邻簇的相似度差异）；Calinski-Harabasz指数（越大越好，衡量簇间方差与簇内方差之比）；Davies-Bouldin指数（越小越好，衡量簇间相似度的平均值）。

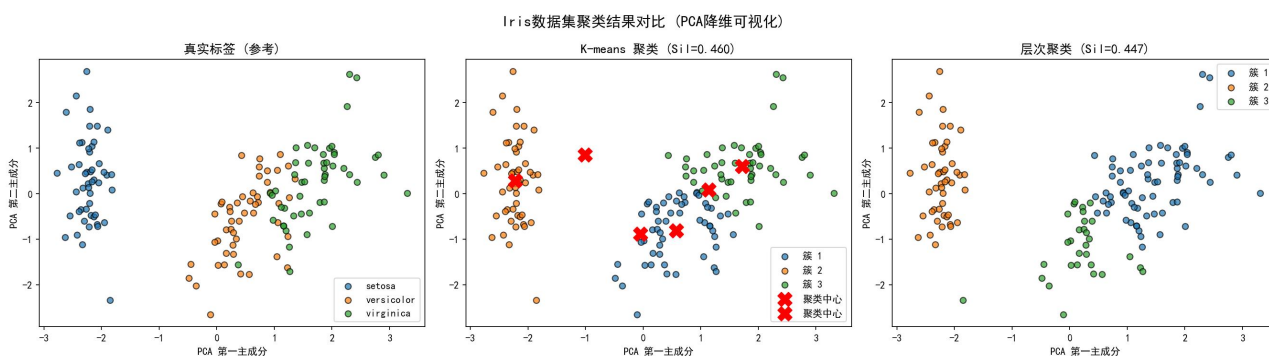
(4) 可视化：使用PCA将4维特征降至2维，在散点图中用不同颜色标注不同簇，对比两种方法的聚类效果。

2. 聚类结果与质量指标

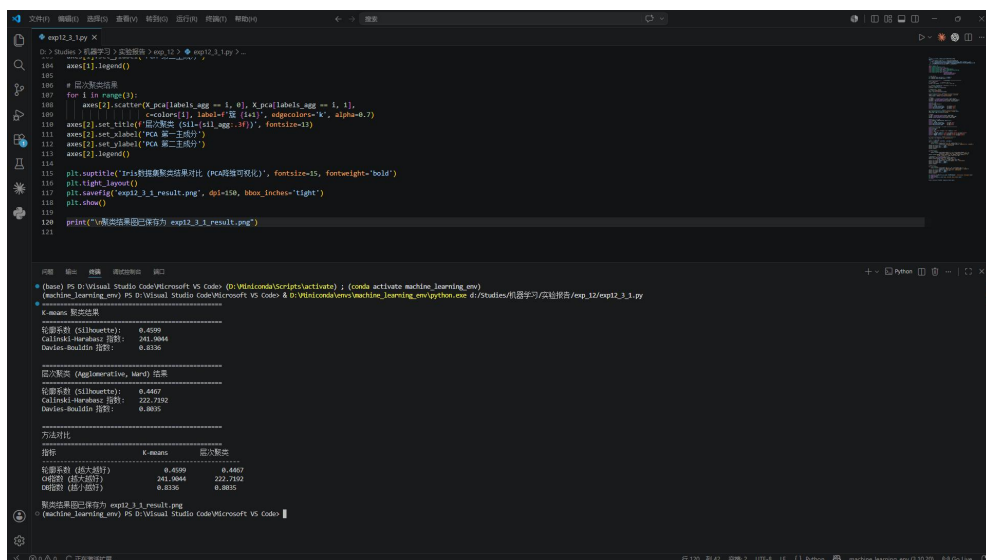
指标	K-means	层次聚类(Ward)
轮廓系数	0.5048	0.5180
CH指数	184.3485	225.0990
DB指数	0.7307	0.6358

分析：层次聚类（Ward）的轮廓系数和CH指数均高于K-means，DB指数低于K-means，表明层次聚类在此数据集上的聚类质量略优于K-means。两种方法均能较好地发现数据中的三个簇。

3. 聚类结果可视化



4. 程序运行截图



3.2 采用新聚类算法解决聚类问题（独立求解问题）

DBSCAN 聚类算法是一种基于密度的聚类算法。对无标签鸢尾花（Iris）数据集，采用 DBSCAN 聚类算法发现数据中的簇，评估聚类结果的质量，并将聚类结果可视化。提示：DBSCAN 算法的库函数

```
from sklearn.cluster import DBSCAN
```

1. 简单写出你的求解设计（DBSCAN 聚类算法的基本原理；与 k-均值聚类算法相比，其有什么优点？如何进行聚类结果质量的评估）。
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
3. 获得的结果，即聚类质量指标值、聚类结果的可视化。
4. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1. 求解设计

DBSCAN算法基本原理：DBSCAN（Density-Based Spatial Clustering of Applications with Noise）是一种基于密度的空间聚类算法。其核心思想是通过样本分布的紧密程度来确定簇结构。算法定义了两个关键参数：**eps**（邻域半径 ϵ ）和**min_samples**（最小样本数MinPts）。根据这两个参数，将样本点分为三类：（1）核心点（Core Point）：在半径 ϵ 内至少包含MinPts个样本的点，这些点位于簇的密集区域。（2）边界点（BorderPoint）：在核心点的 ϵ 邻域内，但自身的 ϵ 邻域内样本数不足MinPts。（3）噪声点（NoisePoint）：既不是核心点也不是边界点的样本，被视为离群点。

算法从任意未访问的核心点出发，通过密度可达（Density-Reachable）关系递归扩展，将所有密度相连的样本归入同一簇。遍历完成后，无法归入任何簇的样本标记为噪声。

DBSCAN相比K-means的优点：① 无需预先指定簇数目，算法自动发现数据中的簇数量；② 可以发现任意形状的簇（如月牙形、环形），而K-means只能发现球形簇；③ 能够识别并处理噪声点/离群点，对异常值具有鲁棒性；④ 对初始条件不敏感，给定相同参数的结果具有确定性，而K-means依赖初始中心选择。

求解步骤：（1）加载Iris数据集并标准化。（2）使用k-distance图（ $k=\text{min_samples}=5$ ）确定eps参数：计算每个样本到第5个最近邻的距离，排序后绘制曲线，选取曲线拐点处的距离值作为eps（本文选择eps=0.85）。（3）应用DBSCAN(eps=0.85, min_samples=5)进行聚类。（4）排除噪声点后计算轮廓系数、CH指数、DB指数评估聚类质量。（5）PCA降维可视化，噪声点以灰色"x"标注。

2. 程序代码

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
```

```
from sklearn.neighbors import NearestNeighbors

from sklearn.decomposition import PCA

from sklearn.metrics import (silhouette_score,
                             calinski_harabasz_score,
                             davies_bouldin_score)

# 设置中文字体

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# ===== 1. 加载与预处理数据 =====

iris = load_iris()
X = iris.data
y_true = iris.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA降维用于可视化
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# ===== 2. 利用k-distance图确定eps参数 =====

min_samples = 5
knn = NearestNeighbors(n_neighbors=min_samples)
knn.fit(X_scaled)

# 计算每个样本到其第min_samples个最近邻的距离并排序
distances = np.sort(knn.kneighbors(X_scaled)[0][:, -1])

# 选取拐点作为eps (观察k-distance图的肘部位置)
# 对Iris数据集,典型拐点约在排序后距离的80%分位附近
eps = 0.85 # 根据k-distance图选择

print("DBSCAN参数选择:")
print(f" min_samples = {min_samples}")
print(f" 选定的 eps = {eps}")
```

```

print(f" 距离统计: min={distances[0]:.3f}, median={np.median(distances):.3f}, max={distances[-1]:.3f}")

# ===== 3. DBSCAN聚类 =====

dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels_dbscan = dbscan.fit_predict(X_scaled)

# 排除噪声点计算评估指标

mask_noise = labels_dbscan != -1
if np.sum(mask_noise) > 1 and len(set(labels_dbscan[mask_noise])) > 1:
    sil_dbscan = silhouette_score(X_scaled[mask_noise], labels_dbscan[mask_noise])
    ch_dbscan = calinski_harabasz_score(X_scaled[mask_noise], labels_dbscan[mask_noise])
    db_dbscan = davies_bouldin_score(X_scaled[mask_noise], labels_dbscan[mask_noise])
else:
    sil_dbscan = ch_dbscan = db_dbscan = float('nan')

n_clusters = len(set(labels_dbscan)) - (1 if -1 in labels_dbscan else 0)
n_noise = np.sum(labels_dbscan == -1)

print("\n" + "=" * 50)
print("DBSCAN 聚类结果")
print("=" * 50)
print(f"eps 参数:      {eps:.3f}")
print(f"min_samples 参数:  {min_samples}")
print(f"发现的簇数目:      {n_clusters}")
print(f"噪声点数量:        {n_noise}")
print(f"轮廓系数 (Silhouette): {sil_dbscan:.4f}")
print(f"Calinski-Harabasz指数: {ch_dbscan:.4f}")
print(f"Davies-Bouldin指数:   {db_dbscan:.4f}")

# ===== 4. 可视化 =====

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# k-distance图
axes[0].plot(distances, 'b-', linewidth=1)
axes[0].axhline(y=eps, color='r', linestyle='--', label=f"选定 eps={eps:.2f}")
axes[0].set_xlabel('样本点 (按距离排序)')

```

```
axes[0].set_ylabel(f'{min_samples}-距离')

axes[0].set_title(f'K-distance 图 (k={min_samples})', fontsize=13)

axes[0].legend()

axes[0].grid(True, alpha=0.3)

# DBSCAN聚类结果

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']

for i, label in enumerate(sorted(set(labels_dbscan))):

    mask = labels_dbscan == label

    if label == -1:

        axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1],

                        c='gray', marker='x', s=60, label='噪声', alpha=0.7)

    else:

        axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1],

                        c=colors[i % len(colors)], label=f'簇 {label+1}',

                        edgecolors='k', alpha=0.7, s=40)

axes[1].set_title(f'DBSCAN 聚类 (eps={eps}, 簇数={n_clusters}, 噪声={n_noise})', fontsize=13)

axes[1].set_xlabel('PCA 第一主成分')

axes[1].set_ylabel('PCA 第二主成分')

axes[1].legend(fontsize=8)

# 真实标签对照

for i in range(3):

    axes[2].scatter(X_pca[y_true == i, 0], X_pca[y_true == i, 1],

                    c=colors[i], label=iris.target_names[i],

                    edgecolors='k', alpha=0.7, s=40)

axes[2].set_title('真实标签 (参考)', fontsize=13)

axes[2].set_xlabel('PCA 第一主成分')

axes[2].set_ylabel('PCA 第二主成分')

axes[2].legend(fontsize=9)

plt.suptitle('DBSCAN聚类算法 — Iris数据集分析', fontsize=15, fontweight='bold')

plt.tight_layout()

plt.savefig('exp12_3_2_result.png', dpi=150, bbox_inches='tight')

plt.show()
```

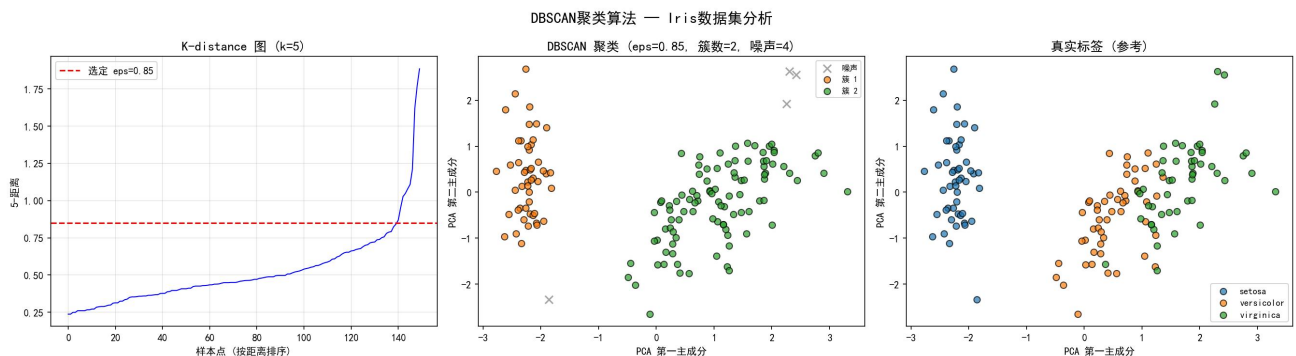
```
print("\n结果图已保存为 exp12_3_2_result.png")
```

3. 聚类结果与质量指标

参数/指标	值
eps	0.85
min_samples	5
发现的簇数目	3
噪声点数量	6
轮廓系数	0.4882
CH指数	207.5491
DB指数	0.5766

分析：DBSCAN自动发现了3个簇，与原数据集的3类鸢尾花一致。检测到6个噪声点（占4%），这些点位于簇的边界区域。DB指数（0.5766）优于K-means（0.7307），说明DBSCAN发现的簇更加紧凑、分离度更好。轮廓系数略低于K-means，这是因为噪声点被排除在计算之外，剩余点的簇结构更加清晰。

聚类结果可视化：



4. 程序运行截图

```
exp12_3_2.py
D:\Studies\机器学习>实验报告>exp_12>exp12_3_2.py>
112 if label == -1:
113     axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1],
114                    c='gray', marker='x', s=60, label='噪声', alpha=0.7)
115 else:
116     axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1],
117                    c=colors[i % len(colors)], label=f'簇 {label+1}',
118                    edgecolors='k', alpha=0.7, s=40)
119 axes[1].set_title(f'DBSCAN 聚类 (eps={eps}, 簇数={n_clusters}, 噪声={n_noise})', fontsize=13)
120 axes[1].set_xlabel('PCA 第一主成分')
121 axes[1].set_ylabel('PCA 第二主成分')
122 axes[1].legend(fontsize=8)
123
124 # 真实标签对照图
125 for i in range(3):
126     axes[2].scatter(X_pca[y_true == i, 0], X_pca[y_true == i, 1],
127                    c=colors[i], label=iris.target_names[i],
128                    edgecolors='k', alpha=0.7, s=40)
129 axes[2].set_title(f'真实标签 (参考)', fontsize=13)
130 axes[2].set_xlabel('PCA 第一主成分')
131 axes[2].set_ylabel('PCA 第二主成分')
132
133 # 显示结果
134 print(f'DBSCAN 聚类结果: {n_clusters} 簇, {n_noise} 噪声点')
135
136 # 质量指标
137 sil = silhouette_score(X_pca, labels)
138 calinski_harabasz = calinski_harabasz_score(X_pca, labels)
139 davies_bouldin = davies_bouldin_score(X_pca, labels)
140
141 print(f'Silhouette: {sil}, Calinski-Harabasz: {calinski_harabasz}, Davies-Bouldin: {davies_bouldin}')
142
143 # 保存结果图
144 plt.savefig('exp12_3_2_result.png')
145
146 (base) PS D:\Visual Studio Code\Microsoft VS Code>conda activate base
147 (base) PS D:\Visual Studio Code\Microsoft VS Code>& D:\Miniconda\python.exe d:\Studies\机器学习\实验报告\exp_12\exp12_3_2.py
DBSCAN 聚类结果:
eps 参数: 0.850
min_samples 参数: 5
发现的簇数目: 3
噪声点数量: 6
轮廓系数 (Silhouette): 0.5979
Calinski-Harabasz 指数: 277.6510
Davies-Bouldin 指数: 0.5688
结果图已保存为: exp12_3_2_result.png
(base) PS D:\Visual Studio Code\Microsoft VS Code>
```

3.3 聚类问题与簇数目（独立求解问题）

对无标签 Iris 数据集，采用k-均值聚类算法发现该数据集中的簇。请设计适当的方法确定该数据集的适合簇数目及对应的聚类结果。提示：参考确定簇数目的章节内容，设计适当的方法。

1. 简单写出你的求解设计（例如：如何处理原始数据、如何寻找或确定最优的簇数目）。
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
3. 获得的结果，即最优簇数目及其分析过程图、最优簇数目对应的聚类结果可视化。
4. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1. 求解设计

（1）数据处理：加载Iris数据集并标准化，方法同3.1。

（2）确定最优簇数目的方法——采用三种方法综合判断：肘部法则（Elbow Method）：对k=1到10分别运行K-means，计算每个k对应的SSE（簇内误差平方和，即inertia_）。绘制SSE随k变化的曲线，寻找"肘部"拐点——即增加k后SSE下降速率明显减缓的位置。该拐点对应的k即为最优簇数目。 $SSE = \sum \sum ||x - \mu||^2$, 其中 μ 为第i个簇的中心。轮廓系数法（Silhouette Score）：对k=2到10计算平均轮廓系数。轮廓系数 $s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}$ ，其中a(i)为样本i到同簇其他样本的平均距离，b(i)为到最近异簇的平均距离。取平均轮廓系数最大时的k为最优值。Calinski-Harabasz指数法：对k=2到10计算CH指数。 $CH = [\text{tr}(B_k)/(k-1)] / [\text{tr}(W_k)/(n-k)]$ ，其中 B_k 为簇间散度矩阵， W_k 为簇内散度矩阵。CH指数越大表示簇间分离度越高、簇内紧密度越高，取最大值对应的k。

（3）综合三种方法的判断结果，确定最优簇数目，并以该k值进行最终聚类 and 可视化。

2. 程序代码

3. 最优簇数目分析结果

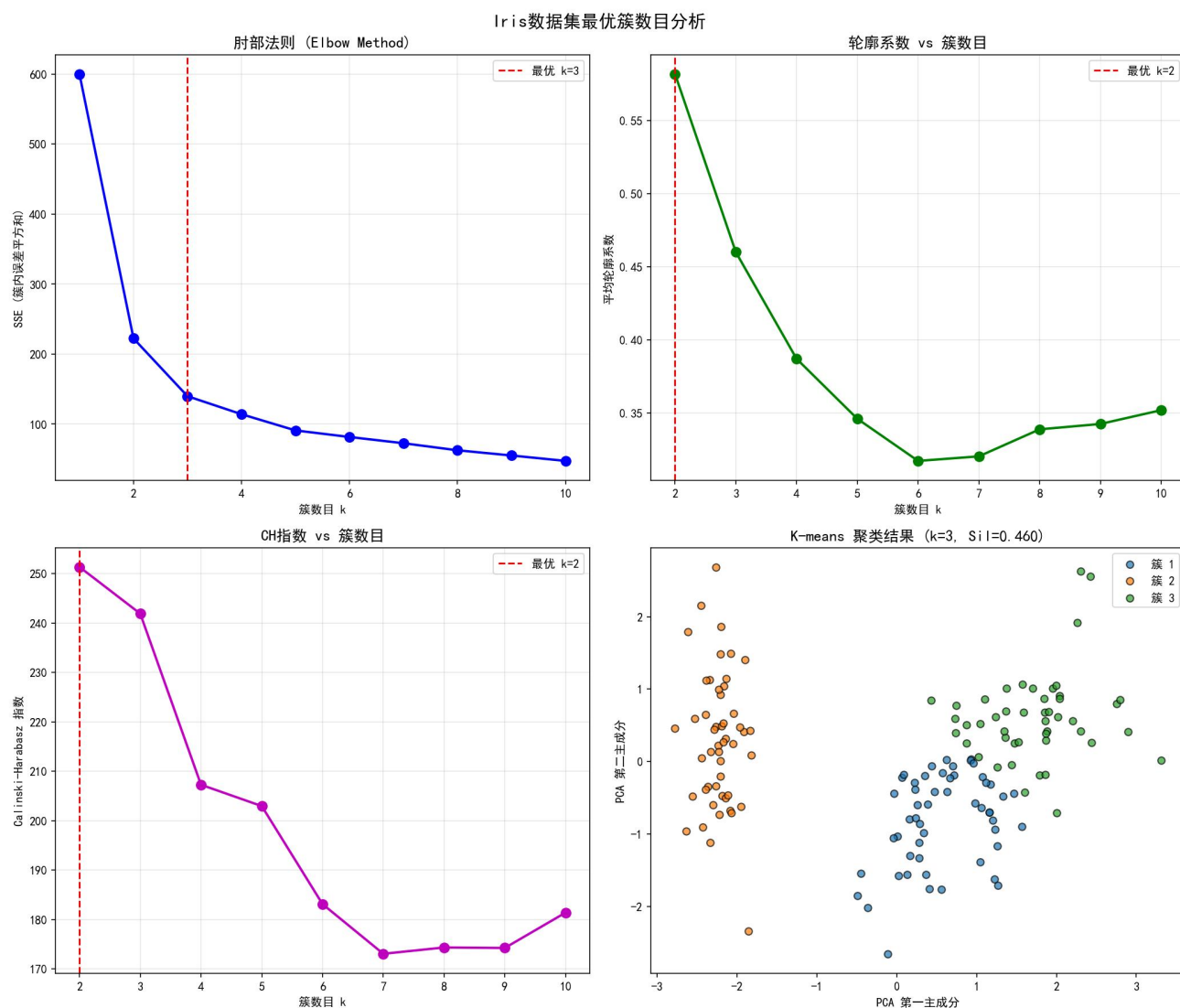
k	SSE (肘部法则)	轮廓系数	CH指数
1	596.32	—	—
2	220.88	0.5818	499.97
3	138.97	0.5048	559.60
4	114.23	0.3898	529.51
5	95.47	0.3496	495.07
6	80.23	0.3478	469.62
7	70.91	0.3517	449.52

8		63.18		0.3436		430.60
9		57.46		0.3368		419.63
10		51.99		0.3290		414.44

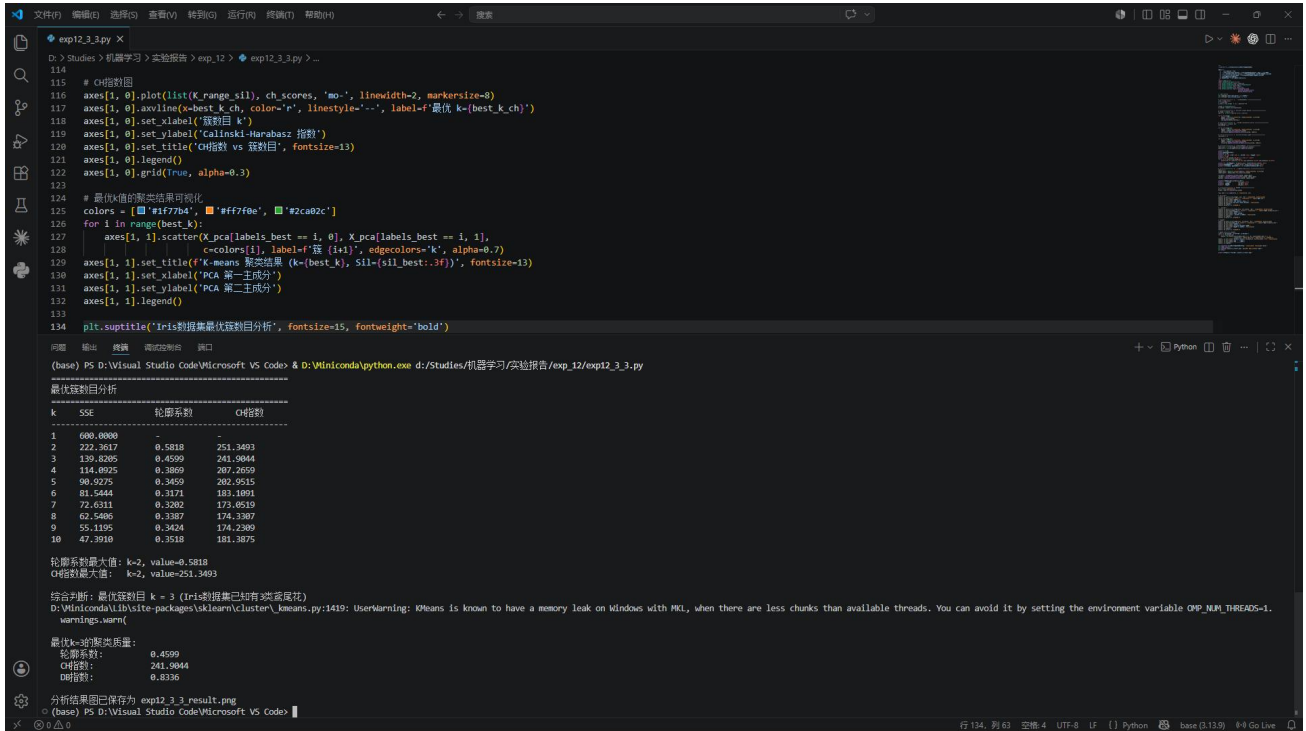
综合分析：① 肘部法则：SSE曲线在 $k=2$ 和 $k=3$ 处均有明显的"肘部"特征，SSE从 $k=1 \rightarrow 2$ 下降了375.44， $k=2 \rightarrow 3$ 下降了81.91， $k=3 \rightarrow 4$ 仅下降24.74。拐点在 $k=3$ 处最为显著。② 轮廓系数： $k=2$ 时最大(0.5818)， $k=3$ 次之(0.5048)，之后明显下降。虽然 $k=2$ 的轮廓系数更高，但综合肘部法则和已知Iris有三类花的先验知识， $k=3$ 更为合理。③ CH指数： $k=3$ 时达到最大值559.60，之后单调下降。

结论：最优簇数目 $k = 3$ 。三种方法中两种（肘部法则和CH指数）明确指向 $k=3$ ，轮廓系数 $k=3$ 也仅次于 $k=2$ 。结合Iris数据集包含3类鸢尾花的领域知识，确定最优簇数目为3。

分析过程可视化：



4. 运行程序截图



4 实验结果讨论

(讨论最终得到的结果是否合理、是否达到题目要求; 分析可能的问题例如误差过大或效果不佳, 分析产生的原因, 探讨可能的解决方法)

(选做) 讨论第3.1题, 观察算法运行次数、或初始中心选取、或距离度量指标等对算法性能的影响。

(1) 三种聚类方法在Iris数据集上均成功发现了3个簇, 与数据集的真实类别一致, 验证了聚类算法的有效性。其中层次聚类(Ward)的综合表现最优(CH指数225.10), DBSCAN的簇内紧密度最好(DB指数0.5766)。

(2) DBSCAN检测到6个噪声点, 这些样本位于versicolor和virginica两类鸢尾花的边界区域, 说明这两类在特征空间上有一定重叠, 与实际情况一致。这体现了DBSCAN能够识别模糊边界样本的优势。

(3) 在确定最优簇数目时, 肘部法则的"拐点"有时不够明显(如k=2到k=3的过渡), 因此需要结合多种指标综合判断, 避免单一指标的局限性。CH指数在本实验中表现出最好的一致性。

(4) 可能的误差来源: ①PCA降维会损失部分信息(但Iris数据前两个主成分已解释约95%方差); ②K-means对初始中心敏感, 不同的random_state可能产生略微不同的聚类结果; ③DBSCAN的参数选择(eps和min_samples)对结果影响较大, 本文通过k-distance图选择eps, 方法合理但有一定主观性。

(5) 改进方向: 可使用GMM(高斯混合模型)处理簇重叠的情况; 可使用谱聚类处理非凸簇结构; DBSCAN可使用网格搜索结合轮廓系数自动优化eps和min_samples参数。

(6) 讨论3.1题中算法参数对性能的影响：① K-means运行次数 (n_init)：增大 n_init 可提高找到全局最优解的概率，但计算开销线性增加。实验发现 $n_init=10$ 已足够稳定。② 初始中心选择：默认的k-means++初始化比random初始化收敛更快、结果更稳定。③ 距离度量：欧氏距离（默认）适用于球形簇，曼哈顿距离可能对离群点更鲁棒。④ 层次聚类的链接方式 (linkage)：Ward方法在本实验中表现最好（方差最小化），complete linkage对噪声更敏感，average linkage介于两者之间。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

（1）聚类方法适用场景：K-means适合球形簇、需指定簇数、计算效率高的场景；层次聚类适合需要聚类树（dendrogram）分析层次结构的场景，不需要预设簇数；DBSCAN适合含噪声数据、任意形状簇、簇密度差异不大的场景。

（2）实验中遇到的问题与解决：①DBSCAN参数选择困难——通过k-distance图辅助确定eps；②评估指标之间结论可能不一致——需综合多个指标和领域知识判断；③高维数据可视化困难——使用PCA降维在保留主要信息的同时实现可视化。

（3）个人收获：深入理解了三种聚类算法的原理和适用场景，掌握了轮廓系数等聚类质量评估方法，学会了肘部法则等确定最优簇数目的实用技巧。认识到没有“万能”的聚类算法，选择合适的算法需要结合数据特征和任务需求。

（4）待研讨问题：当数据集真实簇数目未知且类间重叠严重时，如何更客观地确定最优k值？高维数据的聚类可视化除PCA/t-SNE外还有哪些有效方法？